Static Analysis Report

FoodtruckApp — Sprint 3 — SonarJS and Trivy

Repository	`https://github.com/Unuvert1/FoodtruckApp`
Branch / commit	`feature/sprint-3 @ 5570aa3`
Prepared by	Unubileg (GitHub: Unuvert1)
Report date	October 20, 2026
Scans in this report	SonarJS (code quality & security rules) and Trivy (dependency, secret and misconfiguration scanning)

1. Tools Used

Programming languages

TypeScript throughout — 14,590 lines across 173 `.ts` / `.tsx` files, on Next.js 15 with React 19. The repository also holds SQL migration files (Prisma) and CSS. Node.js 24.19.0, TypeScript 5.9.3.

Sonar — version and edition

Analyser	SonarJS 3.0.7 (`eslint-plugin-sonarjs`), SonarSource's official JavaScript/TypeScript rule set
Rules available	269 rules; the `recommended` profile was enabled
Runner	ESLint 9.39.5 (flat config)
Edition	**Not** SonarQube Server or SonarQube Cloud — the same rule engine, run locally. See the note below.

Why SonarJS and not the SonarQube server. The SonarQube Server and Cloud products run these same SonarJS rules behind a web dashboard. Neither could be run on this machine within the sprint: the server edition needs Docker, and Docker Desktop's engine requires WSL2, which is not installed here and needs administrator rights plus a reboot to add. SonarQube Cloud is free for public repositories like this one, but importing the project needs a GitHub authorization the team has not yet granted. The rules, the rule identifiers (RSPEC numbers) and the issue classifications in this report are therefore the genuine SonarSource ones, executed over the whole codebase; what is missing is the hosted dashboard and its server-side Quality Gate. The gate in section 2 is evaluated locally against the published "Sonar way" conditions and is labelled as such.

Trivy — version and scan type

Version	Trivy 0.76.0 (official Windows x64 build)
Scan type	**Filesystem scan** (`trivy fs`) of the repository root
Scanners enabled	`vuln` (dependency vulnerabilities), `secret` (hard-coded credentials), `misconfig` (infrastructure misconfiguration)
Dependency source	the npm dependency graph resolved from `package-lock.json`
Vulnerability database	downloaded fresh at scan time from the official Trivy DB mirror

Where the scans were executed

Locally, on the developer's Windows 11 machine, against the working tree at commit `5570aa3`. Neither scan runs in CI yet; wiring Trivy into a GitHub Actions workflow is the obvious next step and is noted in section 5.

2. Required Metrics

Sonar

0	0	45	2
BUGS	VULNERABILITIES	CODE SMELLS	SECURITY HOTSPOTS

Metric	Value	Notes
Bugs	**0**	No reliability rule fired across 170 files.
Vulnerabilities	**0**	No injection, hard-coded-credential or crypto rule fired.
Code Smells	**45**	Maintainability issues across 5 rules.
Security Hotspots	**2**	Both are slow-regex (ReDoS) hotspots; both reviewed — see the Sonar report.
Quality Gate	**PASS**	Evaluated *locally* against the published "Sonar way" conditions, not by a Sonar server.

Supporting measures, each produced by a real run rather than an estimate:

Measure	Value	How it was measured
Tests passing	137 of 137 (13 files)	`vitest run`
Line coverage	81.29% (252/310)	`vitest --coverage` (v8). **Covers only the 17 `lib/` modules the tests import**, not the UI layer.
Branch coverage	82.13% (262/319)	same run
Duplicated lines	0.27% (50 lines, 1 clone)	`jscpd` over 270 files

Trivy

0	8	6	0
CRITICAL	HIGH	MEDIUM	LOW

Severity	Count	Notes
CRITICAL	**0**	—
HIGH	**8**	Highest CVSS is 9.1 (postcss). All transitive.
MEDIUM	**6**	Includes 2 in Next.js itself, both fixed in 15.5.27.
LOW	**0**	—
Total	**14**	Every finding is in a third-party package; none is in first-party code.
Hard-coded secrets	**0**	Secret scanner enabled and clean.
Misconfigurations	**0**	No IaC/config files were detected to check.

3. Scope

What was scanned

- The whole application source tree: `app/`, `components/`, `lib/`, `middleware.ts` and the test files — 170 files for Sonar.
- The complete resolved npm dependency graph from `package-lock.json`, production and development, for Trivy.
- The repository tree for hard-coded secrets and for configuration misconfigurations.

What was excluded, and why

Excluded	Reason
`node_modules/`	Installed packages. Dependency vulnerabilities are already covered through the lock file, which is the authoritative list.
`lib/generated/`	The Prisma client is generated code; it is not written or reviewed by us and is recreated on every install.
`.next/`, `out/`, `build/`	Build output.
`.env.local`	Local secrets file. It is git-ignored and has never been committed, so it is not part of the codebase under review; it was excluded so that no credential could be reproduced in this report.
`docs/`, `*.config.*`	Documentation and tool configuration, not application logic.

4. Trend

Baseline sprint — no prior comparison. This is the first sprint in which static analysis was run on FoodtruckApp, so there are no previous issue counts to compare against. The figures in section 2 are the baseline that Sprint 4 will be measured against:

Measure	Sprint 3 baseline	Sprint 4 target
Trivy HIGH	8	0–2 (only those with no upstream fix)
Trivy MEDIUM	6	2
Sonar Code Smells	45	under 25
Sonar Bugs / Vulnerabilities	0 / 0	hold at 0 / 0
Functions over complexity 15	5	0

5. Reflection

The most problematic area is not our own code but the **third-party dependency chain**: all 14 vulnerabilities, including every one of the 8 HIGH findings, come from transitive packages, and two thirds of them trace back to just two entries in `package.json` — the `shadcn` CLI, which is listed as a runtime dependency although it is only ever used as a scaffolding command, and Next.js, which pins an old `postcss` internally. To reduce this next sprint we will move `shadcn` out of `dependencies` and upgrade Next.js to 15.5.27, which together clear 10 of the 14 findings, then add a Trivy step to CI so a vulnerable dependency fails the build instead of waiting for a

sprint-end scan. On the Sonar side the code is clean for reliability and security — 0 bugs and 0 vulnerabilities — so the work there is maintainability: splitting the five functions that exceed the complexity limit (the POS screen and the stop sheet worst of all, at 27 and 23 against a limit of 15) and replacing the 24 nested ternaries, which are the single largest cause of smells.

> **"This static analysis was generated using automated tools during this sprint."**

Static Analysis Report · FoodtruckApp Sprint 3 · commit `5570aa3` · October 20, 2026 · This static analysis was generated using automated tools during this sprint.